CESAR School

INFRAESTRUTURA DE SOFTWARE

Atividade 4

Sincronização entre vagões e passageiros de uma estação de metrô

Implementação em C com pthreads

Júlia Tenório Calado

Recife, novembro de 2026

Sumário

1 Introdução **3**
2 O problema e as restrições **3**
3 Modelo de estado **3**
4 Implementação **4**
4.1 Inicialização 4
4.2 Passageiro esperando o vagão 5
4.3 Aviso de embarque 5
4.4 Carga do vagão 6
4.5 Ausência de sinais perdidos 6
4.6 Atendimento às restrições 7
5 Testes **7**
5.1 Ambiente 7
5.2 Compilação 7
5.3 Execução do runner 7
5.4 Repetição 8
5.5 ThreadSanitizer 8
5.6 Teste adicional 8
6 Dificuldades **9**
6.1 O vagão retornando cedo 9
6.2 Acordar mais passageiros do que assentos 9
6.3 O caso de zero assentos 9
6.4 O sinal perdido 9
6.5 O teste e o tempo 9
6.6 Limitação do TSan no WSL 9
6.7 O que não foi implementado 10
7 Conclusão **10**
Referências **10**

1 Introdução

Esta atividade pede a implementação das funções de sincronização de uma estação de metrô fictícia, a METROREC. A ideia é automatizar o embarque: cada vagão e cada passageiro é uma thread independente, e o vagão só pode partir quando todos os passageiros que estavam esperando já estiverem sentados. O trabalho foi feito em C, com pthreads, e o resultado é o arquivo sync_car.c acompanhado de sync_car.h e de um Makefile.

Além do código, este relatório explica o modelo de estado adotado, o papel de cada variável de condição, os testes executados com o runner fornecido pelo professor e as dificuldades que apareceram no caminho. Todas as decisões foram tomadas pensando nas restrições do enunciado: um único lock por estação, nenhum semáforo, nenhuma espera ocupada e possibilidade de vários passageiros embarcarem ao mesmo tempo.

2 O problema e as restrições

O sistema tem dois tipos de thread. A thread do vagão chama station_load_car(station, count) quando chega à estação com count assentos livres. A thread do passageiro chama station_wait_for_car(station), que só deve retornar quando houver um vagão na estação e um assento livre. Depois de ocupar o assento, o passageiro chama station_on_board(station). O contrato de station_load_car é o mais delicado: ela não pode retornar antes de todos os passageiros estarem sentados e de o vagão estar cheio ou de não haver mais ninguém esperando. Além disso, a estação precisa ser inicializada por station_init.

As restrições do enunciado que influenciaram diretamente o código foram:

- usar apenas mutex e variáveis de condição, sem semáforos nem outras primitivas;
- ter um único lock na struct station;
- permitir que várias threads chamem station_wait_for_car e retornem antes de qualquer chamada a station_on_board;
- não usar espera ocupada;
- assumir no máximo um vagão por vez e todos os passageiros com o mesmo destino.

O runner fornecido pelo professor deixa as expectativas do teste explícitas. Ele cria 100 threads de passageiros e, em seguida, cria vagões com uma quantidade aleatória de assentos entre 0 e 49. Para cada vagão, o programa principal espera que exatamente min(passageiros restantes, assentos livres) threads saiam de station_wait_for_car, chama station_on_board para cada uma e só então verifica se station_load_car retornou. Se o vagão retornar antes disso, o teste acusa “station_load_car returned early”. Se demorar mais de dois segundos, o alarme do teste dispara. Também há verificações de que station_load_car volta imediatamente quando não há passageiros esperando ou quando o vagão tem zero assentos.

3 Modelo de estado

Antes de escrever o código, tentei reduzir o problema a um conjunto pequeno de informações que o vagão e os passageiros precisam compartilhar. A struct station ficou assim:

```
struct station {
    pthread_mutex_t lock;              /* unico lock da estacao */
    pthread_cond_t car_arrived;        /* passageiros esperam pelo vagao */
    pthread_cond_t passenger_boarded; /* vagao espera pelos passageiros */
    int waiting_passengers;            /* passageiros ainda sem assento */
    int free_seats;                    /* assentos livres do vagao atual */
    int boarding_passengers;           /* embarcaram e ainda nao avisaram */
};
```

Tabela 1: campos da struct station.

Campo	Papel
lock	Protege todos os contadores e as duas variáveis de condição.
car_arrived	Fila de espera dos passageiros que ainda não conseguiram assento.
passenger_boarded	Espera do vagão pelos avisos de station_on_board.
waiting_passengers	Quantos passageiros ainda estão tentando pegar assento.
free_seats	Assentos livres do vagão na estação. Zero significa que não há vagão embarcando ou que ele já encheu.
boarding_passengers	Quantos passageiros já reservaram assento mas ainda não chamaram station_on_board.

A ideia é que free_seats funcione como indicador da presença do vagão: se for zero, ou não há vagão na estação, ou o vagão já encheu. waiting_passengers conta apenas quem ainda não conseguiu assento. boarding_passengers conta quem já reservou assento mas ainda não avisou o vagão. Os três contadores são sempre alterados com o mutex travado, então qualquer thread que olhe para eles vê um estado consistente.

A separação entre waiting_passengers e boarding_passengers foi o ponto central do trabalho. Se existisse apenas um contador de passageiros, o vagão não conseguiria distinguir “ainda tem gente para embarcar” de “todo mundo já pegou assento, mas ainda não confirmou”. Essa diferença é justamente o que o runner cobra.

4 Implementação

4.1 Inicialização

station_init apenas zera os contadores e inicializa o mutex e as duas variáveis de condição. Não há destruição dos objetos porque o enunciado não pede e o processo termina depois do teste.

```
void station_init(struct station *station)
{
    station->waiting_passengers = 0;
    station->free_seats = 0;
    station->boarding_passengers = 0;

    pthread_mutex_init(&station->lock, NULL);
    pthread_cond_init(&station->car_arrived, NULL);
    pthread_cond_init(&station->passenger_boarded, NULL);
}
```

4.2 Passageiro esperando o vagão

```
void station_wait_for_car(struct station *station)
{
    pthread_mutex_lock(&station->lock);

    station->waiting_passengers++;

    while (station->free_seats == 0)
        pthread_cond_wait(&station->car_arrived, &station->lock);

    station->free_seats--;
    station->waiting_passengers--;
    station->boarding_passengers++;

    pthread_mutex_unlock(&station->lock);
}
```

O passageiro entra na contagem de waiting_passengers antes de testar os assentos. Isso é importante porque, a partir desse momento, o vagão passa a saber que existe mais alguém para embarcar e não pode partir sem considerar essa thread. O teste dos assentos é feito em um while, e não em um if, por dois motivos: a chamada de pthread_cond_wait pode acordar sem que a condição tenha mudado (spurious wakeup) e um broadcast pode acordar mais passageiros do que assentos disponíveis. Quem não conseguir assento volta a dormir, mas continua contado em waiting_passengers.

Quando o passageiro consegue assento, ele decrementa free_seats, sai de waiting_passengers e entra em boarding_passengers. Tudo isso antes de liberar o lock. Assim, quando o vagão olhar para o estado, ou o passageiro ainda está esperando, ou já está contado como embarcado. Não existe janela em que ele tenha pegado assento sem estar em nenhuma das duas contagens.

4.3 Aviso de embarque

```
void station_on_board(struct station *station)
{
    pthread_mutex_lock(&station->lock);

    station->boarding_passengers--;

    if (station->boarding_passengers == 0)
        pthread_cond_signal(&station->passenger_boarded);

    pthread_mutex_unlock(&station->lock);
}
```

station_on_board só decrementa boarding_passengers. Quando o contador chega a zero, o vagão é acordado com pthread_cond_signal, pois é a única thread que espera nessa variável de condição. O sinal é dado com o mutex ainda travado, o que impede que o vagão teste a condição e volte a dormir no meio da atualização.

4.4 Carga do vagão

```
void station_load_car(struct station *station, int count)
{
    pthread_mutex_lock(&station->lock);

    station->free_seats = count;

    if (count > 0 && station->waiting_passengers > 0) {
        pthread_cond_broadcast(&station->car_arrived);

        while ((station->free_seats > 0 && station->waiting_passengers > 0) ||
               station->boarding_passengers > 0)
            pthread_cond_wait(&station->passenger_boarded, &station->lock);
    }

    station->free_seats = 0;

    pthread_mutex_unlock(&station->lock);
}
```

station_load_car começa gravando count em free_seats. Se count é zero ou não há ninguém esperando, a função retorna imediatamente. Esse caso não é um detalhe: o runner chama station_load_car(&station, 0) com 100 passageiros esperando e espera que a chamada volte na hora, porque um vagão sem assentos não tem como embarcar ninguém. O mesmo vale para o começo do teste, quando ainda não há passageiros.

Quando há assentos e passageiros, o vagão usa pthread_cond_broadcast para acordar todos. Não seria suficiente usar signal: apenas um passageiro acordaria, e o vagão poderia ficar esperando um segundo passageiro que nunca foi acordado. Com broadcast, todos competem pelo lock e cada um pega um assento se ainda houver.

O while que vem depois é a tradução direta do contrato da função. O vagão continua esperando enquanto houver assento livre e alguém ainda esperando (ou seja, o vagão não encheu e ainda tem gente para entrar), ou enquanto houver passageiro que pegou assento e ainda não chamou station_on_board. Quando as duas coisas deixam de valer, todos os passageiros estão sentados e o vagão está cheio ou não há mais ninguém esperando. Antes de sair, free_seats volta a zero para marcar que o vagão deixou a estação; se um passageiro chegar depois disso, ele vê zero e espera o próximo vagão, em vez de pegar um assento que não existe mais.

4.5 Ausência de sinais perdidos

O protocolo usado é o clássico de variáveis de condição: a condição é sempre testada com o mutex travado, e pthread_cond_wait libera o mutex de forma atômica. Um passageiro contado em waiting_passengers está ou dormindo em car_arrived ou segurando o lock. Como o broadcast também é feito com o lock travado, não existe intervalo em que o vagão decida acordar todo mundo e um passageiro entre na contagem sem receber o aviso. O mesmo raciocínio vale para boarding_passengers e passenger_boarded.

4.6 Atendimento às restrições

- Um único lock: a struct tem só um pthread_mutex_t, e todas as funções usam esse mesmo lock.
- Sem semáforos: só aparecem pthread_mutex_* e pthread_cond_*, nenhuma outra primitiva.
- Sem espera ocupada: as duas esperas são feitas com pthread_cond_wait.
- Embarque simultâneo: o broadcast e os contadores permitem que vários passageiros saiam de station_wait_for_car antes de qualquer chamada a station_on_board.
- Um vagão por vez: free_seats é único e é zerado quando o vagão parte.

5 Testes

5.1 Ambiente

Item	Valor
Sistema operacional	Ubuntu 24.04 LTS (WSL2)
Kernel	Linux 6.18.33.2-microsoft-standard-WSL2, x86_64
Compilador	gcc 13.3.0
make	GNU Make 4.3

Tabela 2: ambiente usado nos testes.

Os testes foram feitos no Ubuntu dentro do WSL2 porque a máquina usada para desenvolver roda Windows. O código não usa nada específico de Linux além de pthreads, então deve compilar em qualquer sistema POSIX com um compilador C adequado.

5.2 Compilação

O Makefile usa gcc com -Wall -Wextra -O2 -pthread e gera o executável sync_car. A saída de make foi:

```
$ make clean && make
rm -f sync_car.o sync_car-runner.o sync_car
gcc -Wall -Wextra -O2 -pthread -c sync_car.c -o sync_car.o
gcc -Wall -Wextra -O2 -pthread -c sync_car-runner.c -o sync_car-runner.o
gcc -Wall -Wextra -O2 -pthread -o sync_car sync_car.o sync_car-runner.o
```

O único warning é do arquivo sync_car-runner.c, no parâmetro não usado de alarm_handler. Ele já vem do arquivo do professor e não foi alterado. O sync_car.c compila sem avisos.

5.3 Execução do runner

Uma execução completa do runner produziu a saída abaixo, resumida:

```
car entering station with 39 free seats
car departed station with 39 new passenger(s) (expected 39)
car entering station with 35 free seats
car departed station with 35 new passenger(s) (expected 35)
car entering station with 12 free seats
car departed station with 12 new passenger(s) (expected 12)
car entering station with 28 free seats
car departed station with 14 new passenger(s) (expected 14)
Looks good!
```

O runner sorteia os tamanhos dos vagões a cada execução, então a sequência muda. No exemplo, o último vagão tinha 28 assentos e levou os 14 passageiros que faltavam; a função retornou sem esperar o vagão encher, como manda o enunciado.

5.4 Repetição

Testes com threads podem passar por sorte. Para reduzir esse risco, executei o runner 700 vezes, em duas rodadas, sem nenhuma falha. A tabela abaixo registra as rodadas.

Rodada	Execuções	Falhas
1	200	0
2	500	0
Total	700	0

Tabela 3: execuções do runner.

Também compilei com -std=gnu99 -Wall -Wextra -pedantic e o build passou, novamente com o único warning do runner.

5.5 ThreadSanitizer

Compilei o runner junto com a implementação usando -fsanitize=thread. No WSL, o TSan abortava com “unexpected memory mapping” por causa da randomização de endereços; rodando com setarch x86_64 -R, que desliga o ASLR, ele funcionou. Foram cinco avisos, todos no sync_car-runner.c: leituras e escritas das variáveis volatile load_car_returned e threads_completed, que são do próprio teste, e threads que não recebem join. Nenhum aviso apontou para sync_car.c. Isso não prova ausência de erro, mas indica que o problema não estava na implementação.

Os casos de borda também aparecem no runner: ele chama station_load_car com zero assentos antes de criar os passageiros, chama de novo com zero assentos depois de criar as 100 threads, e em várias execuções o último vagão tem mais assentos do que passageiros restantes. Nos três casos o comportamento esperado é o mesmo: o vagão parte sem encher, mas só depois de todos os que embarcaram terem confirmado.

5.6 Teste adicional

Além do runner, escrevi um teste próprio para exercitar o uso mais natural da interface: cada thread de passageiro chama station_on_board logo depois de station_wait_for_car, sem a intervenção da thread principal. O teste cria 200 passageiros e vários vagões com 0 a 19 assentos, e verifica, a cada vagão, se embarcou exatamente min(restantes, assentos). Rodei

esse teste 100 vezes, sem falhas. Ele não faz parte da entrega porque o runner do professor já cobre o contrato, mas serviu para confirmar que a implementação não depende de quem chama station_on_board.

6 Dificuldades

6.1 O vagão retornando cedo

Minha primeira versão controlava apenas os assentos: o vagão acordava os passageiros, esperava free_seats chegar a zero e retornava. O runner reprovou com “station_load_car returned early”. O motivo é que reservar o assento não significa que o passageiro já avisou o vagão; no teste, station_on_board é chamado pela thread principal depois que a thread do passageiro retorna. A solução foi criar boarding_passengers e só deixar o vagão sair quando esse contador zera. Foi a mudança que mais impactou o código.

6.2 Acordar mais passageiros do que assentos

Como o vagão usa broadcast, vários passageiros acordam ao mesmo tempo e só alguns encontram assento. No começo eu tinha medo de que os que não conseguissem assento perdessem a vez ou fossem contados duas vezes. O while em volta do cond_wait resolve isso: quem não pega assento volta a dormir ainda contado em waiting_passengers, e quem pega assento sai da contagem e entra em boarding_passengers. A chave foi manter as duas transições dentro do mesmo bloco protegido pelo mutex.

6.3 O caso de zero assentos

É fácil tratar count == 0 como um vagão normal e acabar esperando passageiros que nunca vão embarcar. O runner chama station_load_car(&station, 0) com 100 passageiros esperando e exige retorno imediato. A condição count > 0 antes do broadcast resolve, mas foi preciso prestar atenção porque o caso não aparece no uso real.

6.4 O sinal perdido

Em uma versão intermediária eu sinalizava o vagão fora da seção crítica. Isso abre uma janela: o vagão pode acordar, testar a condição antes do decremento e voltar a dormir, e o sinal se perde. Passei o decremento e o signal para dentro do mutex. Como o cond_wait libera o mutex atomicamente, o vagão não consegue testar a condição no meio da atualização.

6.5 O teste e o tempo

O runner assume que, quando o vagão chega, todos os passageiros restantes já estão esperando. Isso nem sempre é verdade logo depois da criação das 100 threads, mas na prática elas entram em station_wait_for_car em poucos milissegundos e o problema não apareceu nas 700 execuções. Não dá para eliminar essa corrida dentro da implementação, porque um passageiro que ainda não chamou station_wait_for_car não está esperando e o enunciado permite que o vagão parta sem ele.

6.6 Limitação do TSan no WSL

O ThreadSanitizer não rodava de primeira e abortava com “unexpected memory mapping”.

Depois de pesquisar, vi que é um problema conhecido do WSL com ASLR e resolvi com setarch -R. Os avisos que sobraram são do runner, não da implementação.

6.7 O que não foi implementado

Não implementei prioridade ou ordem justa entre passageiros. O enunciado não pede e a ordem é decidida pelo escalonador. Também não criei uma função de destruição da estação, porque o teste não chama e o processo termina em seguida; se a estação fosse reaproveitada em outro programa, seria necessário destruir o mutex e as variáveis de condição. Por fim, a implementação assume um vagão por vez, como o enunciado autoriza; com dois vagões simultâneos, free_seats seria compartilhado e o controle deixaria de valer.

7 Conclusão

O resultado atende a todas as restrições do enunciado e passou no runner do professor em 700 execuções. A parte mais difícil não foi usar pthread_cond_wait, mas definir exatamente qual informação o vagão precisa para saber que pode partir. Separar passageiros esperando de passageiros que já pegaram assento mas ainda não confirmaram foi o que tornou o contrato de station_load_car implementável com um único lock. Também ficou claro por que o broadcast e o while em volta do cond_wait são necessários: sem eles, ou o vagão acorda poucos passageiros, ou perde avisos.

Referências

BUTENHOF, David R. Programming with POSIX Threads. Reading: Addison-Wesley, 1997.

IEEE. Standard for Information Technology: Portable Operating System Interface (POSIX). IEEE Std 1003.1-2017, 2017.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

THE Open Group. pthread_cond_wait(3), pthread_cond_broadcast(3) e pthread_mutex_lock(3). Manual do POSIX. Disponível nas páginas de manual do Linux.

Enunciado da Atividade 4 de Infraestrutura de Software (METROREC).